Haul

github.com/Gabriel-Castiglia/haul

Gabriel Castiglia · November 11, 2026 · version 1.5.0 · GPL-2.0-only · built and tested with yt-dlp 2026.08.19

English · Español

Windows version: https://github.com/Gabriel-Castiglia/haul-windows

> **macOS 15 or later on Apple silicon (arm64) only.** There is no build for Macs with an Intel processor.

PDF manual: `docs/Haul-manual-en.pdf`

Haul is a macOS app that works as a front end for yt-dlp. The app builds the command, shows the progress and manages a download queue. The actual work is done by yt-dlp, bundled with the app at a fixed, verified version, and by ffmpeg, installed with Homebrew. The interface is available in English and Spanish, selectable from the Language menu; it starts in English on first launch and remembers the choice. The window can be light, dark or follow macOS, in one of twelve colours.

Requirements

- macOS 15 or later, on a Mac with Apple silicon. Tested on macOS 27.
- Homebrew in `/opt/homebrew`.
- To build from source: Apple's Command Line Tools (`xcode-select --install`). Xcode is not needed.

yt-dlp ships with the app. Python, deno and ffmpeg are installed by the installer, so nothing has to be installed by hand.

Installation

With the installer package

Download `Haul-<version>.pkg` from the latest release and open it. The installer is in English or Spanish, following the language of the Mac. It checks that Homebrew is installed and stops with a message if it is not; then it installs `Haul.app` in `/Applications`, quitting a running copy first, and installs Python, deno and ffmpeg with Homebrew or upgrades them if they are already there. The first time that step can take a few minutes. Installing a newer package updates Haul and keeps its settings.

The package is not signed with an Apple Developer ID, so the first time macOS refuses to open it ("Apple could not verify…"). Open System Settings → Privacy & Security, scroll to the message about `Haul-<version>.pkg` and click Open Anyway, then confirm. The installed app opens normally. The `SHA256SUMS.txt` file of each release lets you check the download first:

```
shasum -a 256 Haul-1.5.0.pkg
```

To uninstall, quit Haul and drag `/Applications/Haul.app` to the Trash. Python, deno and ffmpeg stay, because other programs may use them; `brew uninstall` removes them.

From source

```
git clone https://github.com/Gabriel-Castiglia/haul.git
cd haul
scripts/instalar.sh
```

The installer:

1. installs Python, deno and ffmpeg with Homebrew or, if they are already present, upgrades them;
2. builds the app, checks the bundled yt-dlp against its official checksums, and signs the app with the local identity (see "Signing and permissions");
3. copies it to `/Applications/Haul.app`, quitting it first if it is running.

Running it again is how everything gets updated, dependencies and app alike. `scripts/paquete.sh` builds the installer package instead, in `build/`. A copy installed by the package belongs to root, so `scripts/instalar.sh` stops and says how to remove it first.

Usage

1. Paste one or more links into the box at the top. Links can be pasted on their own, one per line, or inside text copied from somewhere else; the app extracts them and drops duplicates. If clipboard access is allowed, the app picks up a copied link by itself when it opens.
2. Choose the format: Video (MP4) or Audio only (MP3).
3. Choose the destination folder. The first time it is Downloads; after that the app keeps the last folder chosen, and the format too.
4. Press Download or ⌘↩. The box empties when the download starts, and a link already in the queue is not picked up again from the clipboard.

Links go into a queue and are downloaded one at a time. More links can be added while the queue runs. Each row shows the thumbnail, title, percentage, speed and time remaining, and once finished it offers to reveal the file in the Finder or open it. Failed or cancelled downloads can be retried or removed. A cancelled download deletes the partial files it had created; files that were already in the folder, such as an earlier download of the same video in another format, are never touched.

Any row can be paused with its pause button and continued with its play button; the partial files stay on disk meanwhile, so the download picks up where it stopped. "Pause all", above the queue, holds the whole queue: the running download stops and waits first in line until "Resume all". "Retry all" appears when some download failed and starts every failed one again. Rows can be dragged with the mouse to change their order, and downloads start in the order of the list.

A download that fails is tried again up to four times, two seconds apart, and the row shows which attempt is running ("Retry 2/4"). Partial files are kept between attempts, so yt-dlp resumes where it stopped. When a download finishes, the app also deletes the intermediate files yt-dlp could not remove by itself, such as the original audio of an MP3 while a file sync client held it open.

The "Clear finished automatically" checkbox, below the format, is always in view, even with an empty queue. While it is checked, every download that finishes or is cancelled leaves the list by itself, and the summary keeps counting it; failed ones stay, so they can be retried. Checking it also clears what had already finished. The app remembers the setting.

Videos with "instrumental" in the title wait in the queue, paused, unless "Download instrumentals" is checked. A paused row can be downloaded on its own with its download button, and checking the option resumes all of them.

The ⓘ button in the window, or Haul > About Haul in the menu bar, shows the version, the meaning of the name, the author, the license and the link to this repository, in the interface language. Everything from the header down to the Download button stays fixed; only the queue scrolls, so adding links never moves the window.

The app can also be started with links from Terminal or from Shortcuts:

```
open -a Haul --args --descargar URL1 URL2
```

Formats

Format	Result	yt-dlp options
Video	MP4 with H.264 and AAC, at the highest resolution available in H.264	`-f bv*+ba/b -S vcodec:h264,res,acodec:aac --merge-output-format mp4`
Audio only	MP3 at the highest quality	`-x --audio-format mp3 --audio-quality 0`

H.264 is preferred because QuickTime and iOS play it without conversion. Files are named after the video title.

Playlists

A playlist link or a channel page is read first, without downloading anything, and every video becomes its own row in the queue, with its thumbnail and its position ("Mix · 3/12"). The files go into a subfolder named after the playlist and are numbered in playlist order: `01 – Title.mp4`, `02 – Title.mp4`, and so on. Each video can be cancelled or retried on its own. A playlist with more than 200 videos asks for confirmation first.

A link to a video that was opened from a playlist brings the whole playlist while "Download whole playlists" is checked, as it is by default. With the option unchecked, a card in the window asks each time whether to download only that video or the whole playlist. The app remembers the setting. Other links download as they are, one video each.

Some tag pages that group several posts, each with an embedded video, are read the same way: one row per post, from the oldest to the newest, so a series keeps its episode order.

Repository layout

Path	Contents
`fuentes/Haul.swift`	App source code (SwiftUI, a single file)
`assets/`	The engraving the icon comes from
`scripts/make_icon.py`	Draws the icon in its twelve colours (Python 3 with numpy, opencv-python, scipy and Pillow); the same script makes the icon of the Windows version
`icono/icono-1024.png`	The app icon, on its default orange, source for every size
`icono/accents/`	The icon on each colour, for the Dock
`docs/images/`	Pictures of the window
`scripts/instalar.sh`	Dependencies, build and installation
`scripts/compilar.sh`	Build and signing into `build/`
`scripts/paquete.sh`	Builds the installer package `build/Haul-<version>.pkg`
`instalador/`	Package definition: `Distribution.xml`, the scripts that quit Haul and install the Homebrew packages, and the pages in English and Spanish
`scripts/firma.sh`	Local signing identity (sourced by `compilar.sh`)
`scripts/manual.sh`	Builds the PDF manuals from the READMEs
`scripts/actualizar-yt-dlp.sh`	Replaces the bundled yt-dlp with another verified release
`plugins/haul/`	Haul's own yt-dlp extractor plugin, for links the bundled version does not handle
`vendor/yt-dlp/`	Bundled copy of yt-dlp 2026.08.19, with sources, checksums, signature and licenses
`docs/Haul-manual-en.pdf`, `docs/Haul-manual-es.pdf`	PDF manuals, in English and Spanish
`docs/manual.css`	Stylesheet for the manuals
`README.md`, `README.es.md`	This documentation, in English and Spanish
`CHANGELOG.md`	Version history

The `build/` folder is generated at build time and is not part of the repository.

Signing and permissions

macOS stores the permissions a user grants, such as access to Downloads or to the clipboard, together with the app's signature. An ad hoc signature changes on every build, so macOS asks for them again. That is why the project signs with its own stable certificate, "Haul (local signing)", which `scripts/firma.sh` creates the first time on the Mac where the app is built:

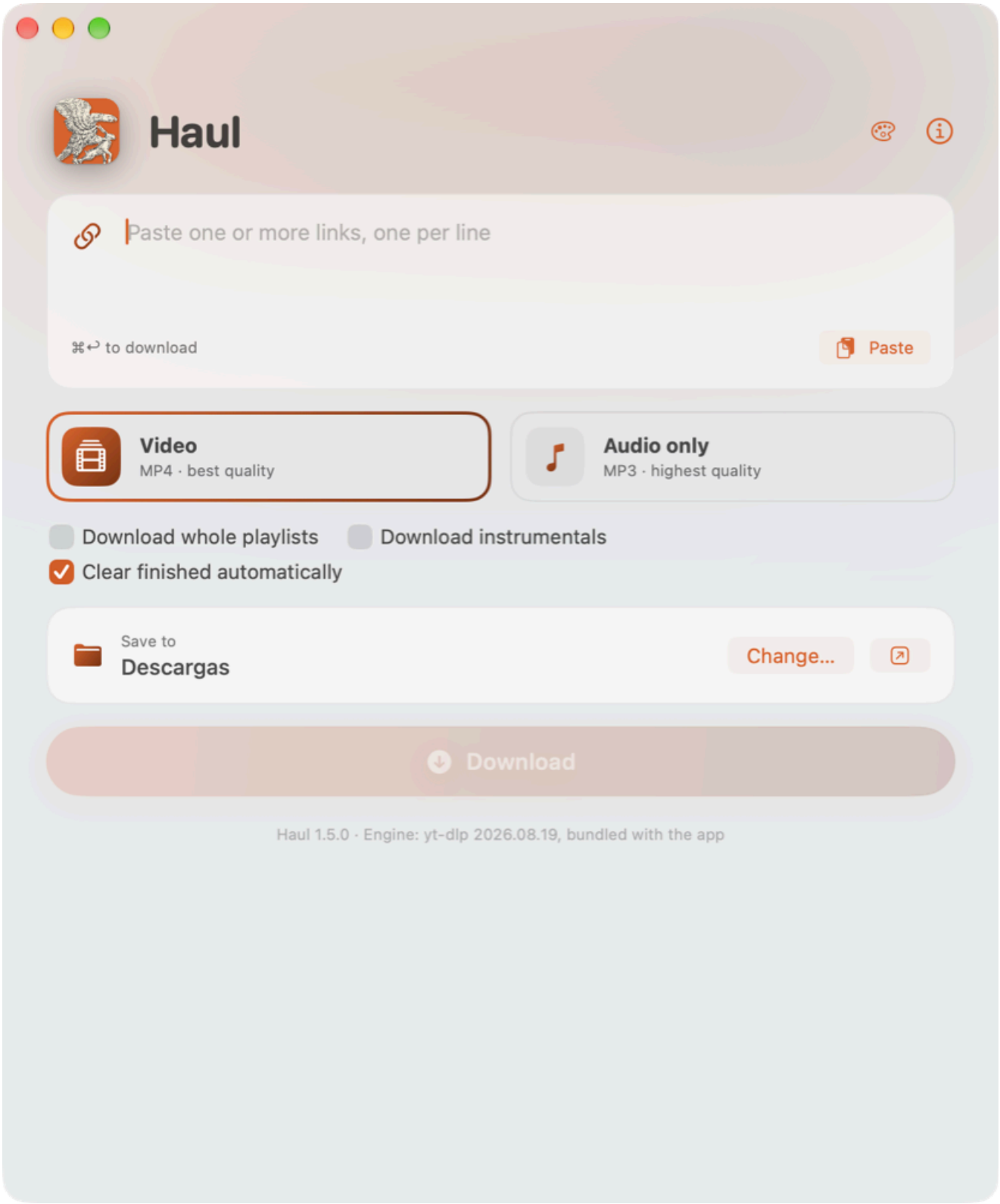
- the certificate and its key live in a dedicated keychain, `~/Library/Keychains/haul-signing.keychain-db`;
- that keychain's password lives in the login keychain, under the name `haul-local-signing`.

With that signature, permissions granted once are kept across versions. The certificate exists only on the Mac where it was created. The app inside the installer package carries the signature of the Mac that built the release, the same in every release, so permissions are kept across updates installed from packages too.

Automatic pasting of links when the app opens works only if clipboard access is set to "Always Allow" (System Settings, Privacy & Security, Paste from Other Apps). Without that permission the app reads the clipboard only when Paste is pressed, and shows no prompt when it opens.

Appearance and icon

The Appearance menu, also behind the palette button of the window, chooses light, dark or as macOS, and one of twelve colours. The colour chosen is used for the buttons, check boxes and selected items, and for the icon in the Dock while Haul is open: the eagle flies on a tile of that colour. The icon in the Finder and in Launchpad stays orange, because changing it would break the app's signature. A Haul that already ran keeps the dark window it had; a new installation follows macOS.



The icon is the eagle carrying a hare from the engraving "Eagle bringing a young woman a hare" by the Monogrammist B (Italy, 17th century), Rijksmuseum, public domain (CC0). `scripts/make_icon.py` cuts it out of the scan in `assets/` and draws it on each colour; it needs Python with numpy, opencv-python, scipy and Pillow. After drawing it again, `scripts/instalar.sh` reinstalls the app. If the Dock keeps showing the old icon, remove the app from the Dock and add it again.

New versions

Once a day Haul asks GitHub for the number of the latest version. If it is newer, a message says so and offers to open the download page; Haul never downloads or installs a new version by itself. The request carries nothing but the program name and its version. About Haul turns it off ("Tell me when there is a new version") and asks at any time ("Check for a new version").

Maintenance

Haul is built and tested with yt-dlp 2026.08.19, and that exact copy ships inside the app. A pinned version keeps the behavior stable and does not depend on the yt-dlp repository staying online. The trade-off is that sites change often, and sooner or later a fixed version stops working with them. When that happens, a new release of Haul moves to a newer yt-dlp:

```
scripts/actualizar-yt-dlp.sh <version>
scripts/instalar.sh
```

The first script downloads the release, verifies its signature with the key stored in `vendor/yt-dlp/public.key` and its SHA-256 checksums, and only then replaces the files. Running `scripts/instalar.sh` again also upgrades Python, deno and ffmpeg.

The most common yt-dlp errors are shown in plain words in the interface language (video unavailable, private video, unsupported site, no connection); anything else appears exactly as yt-dlp prints it.

Preferences (language, folder, format, window size, "Clear finished automatically" and "Download instrumentals") are stored in the `io.github.gabriel-castiglia.haul` domain. They can be reset with `defaults delete io.github.gabriel-castiglia.haul`.

Technical notes

The app runs the bundled yt-dlp with Homebrew's Python as a child process and adds `/opt/homebrew/bin` to `PATH`, because an app launched from the Finder does not inherit the Terminal's and would not find deno or ffmpeg. To avoid depending on the text yt-dlp prints for the Terminal, it asks for output in a custom, tab-separated format:

Line	Contents	Source
`META`	id, channel, uploader, title and thumbnail, checked before downloading	`-O pre_process:…`
`INFO`	title, uploader, duration, thumbnail and file name	`-O before_dl:…`
`AVANCE`	percentage, speed and time remaining	`--progress-template`
`ARCHIVO`	final file path	`-O after_move:…`

Paused instrumentals are held back with `--match-filters`, so yt-dlp downloads nothing until they are allowed.

Some links fail inside yt-dlp itself, its latest release included: a video page that now asks for an account although its embedded player does not, and pages whose data moved to a different place. Haul ships a yt-dlp extractor plugin of its own in `plugins/haul/`, copied into `Contents/Resources/plugins` and passed with `--plugin-dirs`. Plugin extractors are tried before the built-in ones, and each one only matches the links it fixes. When the embedded player answers with an occasional 401 error, the plugin retries up to three times. Python runs with `PYTHONDONTWRITEBYTECODE=1` so it does not write `__pycache__` inside the signed app.

Building without Xcode imposes two limitations, both handled in the code. First, SwiftUI's `@State` macro is unavailable, because its plugin ships only with Xcode; the code uses `@StateObject` with the `Marca` class instead. Second, with a hidden title bar SwiftUI ignores `.defaultSize`, so the initial window size is set through AppKit in `applicationDidFinishLaunching`.

Credits

Haul is a front end; the hard work belongs to these projects:

Project	Role in Haul	License
yt-dlp, by the yt-dlp contributors, fork of youtube-dl	Download engine, bundled in `vendor/yt-dlp`	Unlicense
yt-dlp-ejs, included in the yt-dlp executable	JavaScript challenge solver	Unlicense
meriyah, by KFlash and others, inside yt-dlp-ejs	JavaScript parser	ISC
astring, by David Bonnet, inside yt-dlp-ejs	JavaScript code generator	MIT
FFmpeg, installed with Homebrew, not bundled	Merges picture and sound, converts to MP3	LGPL-2.1+ / GPL-2.0+, depending on the build
Deno, installed with Homebrew	JavaScript runtime used by yt-dlp	MIT
Python, installed with Homebrew	Runs yt-dlp	PSF License
Homebrew	Installs the dependencies	BSD-2-Clause
pandoc, only to build the PDF manuals	Markdown to HTML conversion	GPL-2.0+

The license texts of the bundled components are in `vendor/yt-dlp/licenses/` and are copied into the app, under `Contents/Resources/licenses/`.

License

Copyright (C) 2026 Gabriel Castiglia.

Haul is free software, released under the GNU General Public License, version 2 (GPL-2.0-only), the same license as the Linux kernel. It may be used, studied, modified and redistributed; any modified version that is distributed must be released under the same license, together with its source code. The full text is in `LICENSE`.

Haul includes an unmodified copy of yt-dlp in `vendor/yt-dlp`, which keeps its own licenses (Unlicense, with ISC and MIT components) and is not covered by the GPL. Haul runs yt-dlp and ffmpeg as separate programs and does not link to them; ffmpeg is not bundled.